

UI自动化使用手册

时间：2026-06-08

快速入门

1. 完成项目部署和服务启动

 UI自动化部署指南

2. 脚本录制

2.1 统一录制入口

[record.py](#)

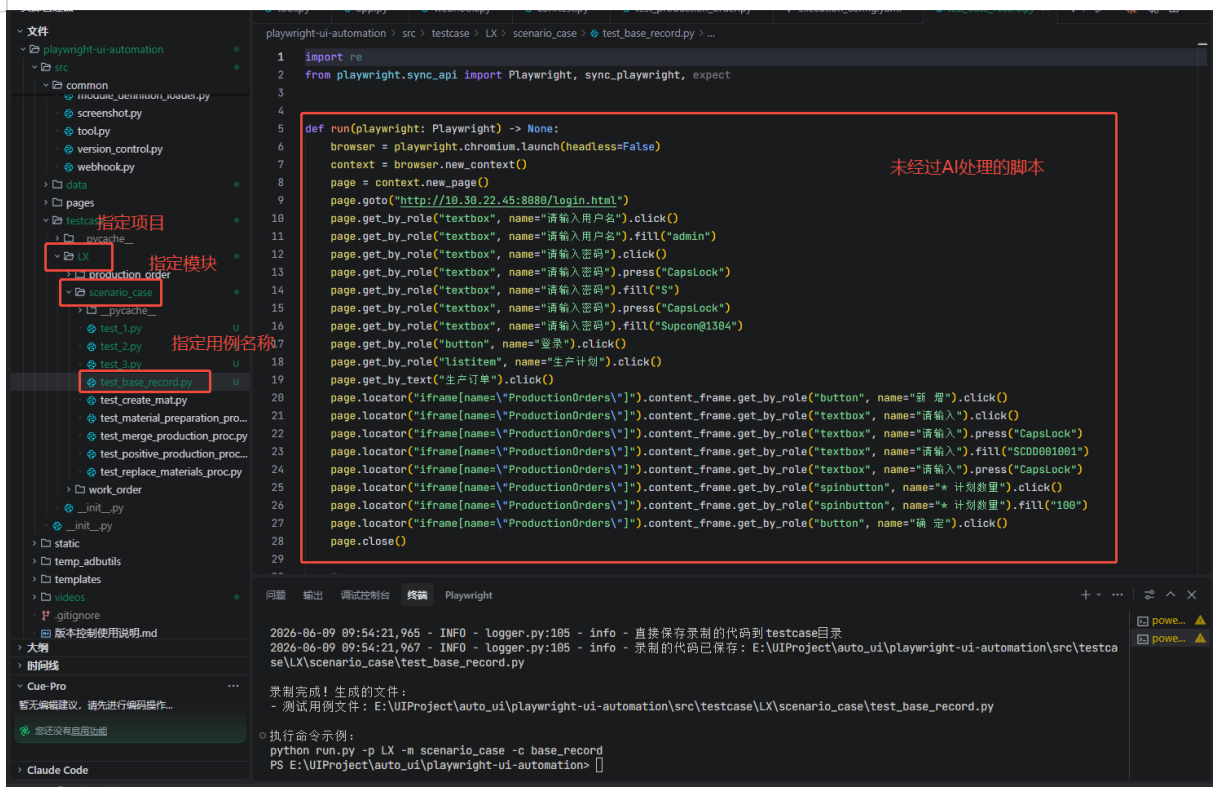
2.2 参数解析

- 1 --project 必填参数 代表项目名 简写'-p'
- 2 --module 必填参数 代表模块名 简写'-m'
- 3 --case 必填参数 代表用例名 简写'-c'
- 4 --url 非必填参数 代表被测网站地址（可选，不传则从配置文件获取）简写'-u'
- 5 --browser 非必填参数 代表浏览器类型，默认chromium 简写'-b'
- 6 --output 非必填参数 代表输出路径 简写'-o'
- 7 --format 非必填参数 代表是否AI格式化代码，0-否，1-是 简写'-f'
- 8 --layering 非必填参数 代表是否AI分层代码，0-否，1-是 简写'-l'
- 9 --ms_id 非必填参数 代表用例ms的id 简写'-ms'

2.3 参数示例

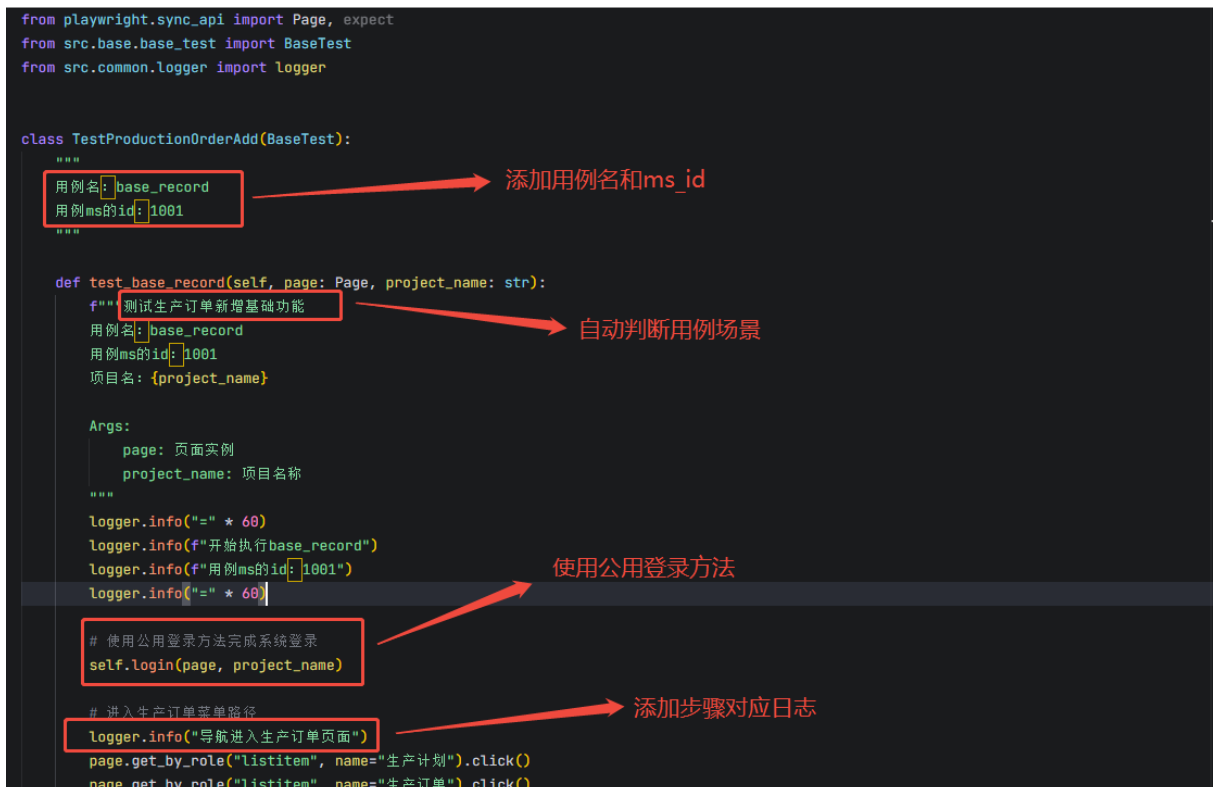
- 1 # 基本录制（不使用AI处理）
- 2 python record.py -p LX -m scenario_case -ms 1001 -c base_record -u
http://10.30.22.45:8080/

基本录制结果展示



- # AI格式化录制
- python record.py -p LX -m scenario_case -ms 1001 -c base_record -u http://10.30.22.45:8080/ -f 1

AI格式化录制结果展示



- 1 # AI格式化分层录制
- 2 python record.py -p LX -m scenario_case -ms 1001 -c base_record -u
http://10.30.22.45:8080/ -f 1 -l 1

AI格式化分层录制结果展示

The screenshot displays a code editor with a project structure on the left and code on the right. The project structure is organized into layers: **src**, **common**, **data**, **pages**, and **testcase**. The **data** layer contains `data_base_record.py`, which is highlighted with a red box and an arrow pointing to it from the text "data层, 保存用例的数据, 如订单号, 计划数量". The **pages** layer contains `page_base_record.py`, which is also highlighted with a red box and an arrow pointing to it from the text "page层保存页面元素".

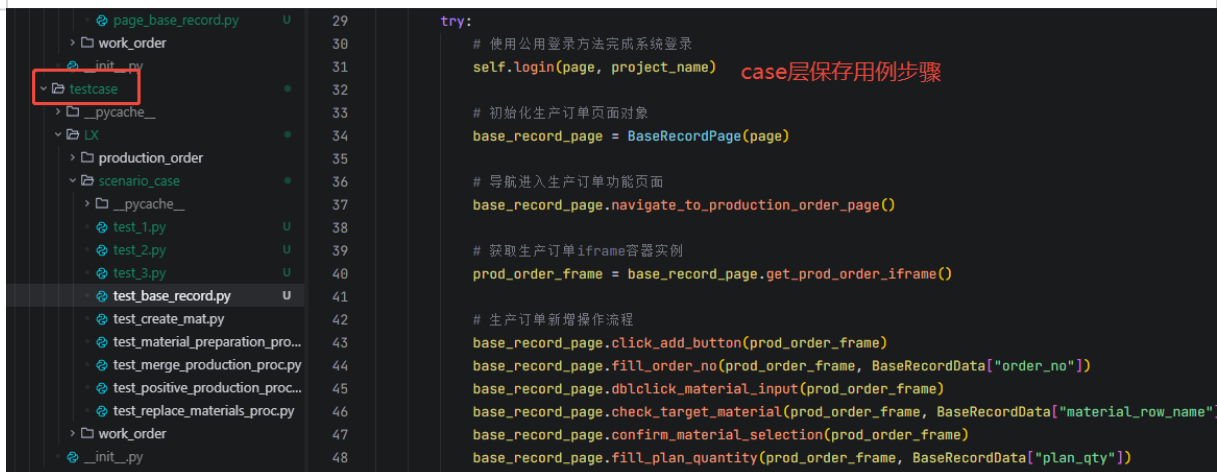
The code in `data_base_record.py` defines a `BaseRecordData` dictionary:

```
BaseRecordData = {
    "order_no": "SC0D555444554",
    "plan_qty": 100,
    "material_row_name": "1 MAT_1780895973_GWFZ 物料_GWFZ"
}
```

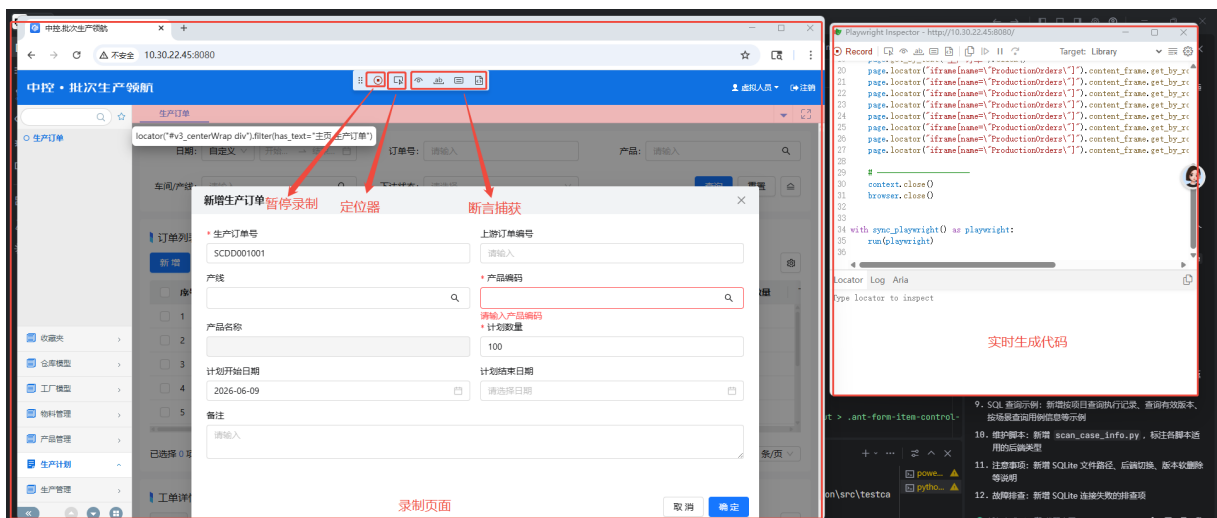
The code in `page_base_record.py` defines a `BaseRecordPage` class with methods for navigating to the production order page:

```
class BaseRecordPage:
    def __init__(self, page: Page):
        self.page = page
        # 元素定位器集中管理
        self.menu_production_management = page.get_by_role("listitem", name="生产管理")
        self.menu_production_plan = page.get_by_role("listitem", name="生产计划")
        self.menu_production_order = page.get_by_role("listitem", name="生产订单")
        self.entry_production_order = page.get_by_text("生产订单")
        self.prod_order_iframe_locator = "iframe[name='ProductionOrders']"

    def navigate_to_production_order_page(self):
        """导航进入生产订单功能页面"""
        logger.info("点击左侧导航菜单【生产管理】")
        self.menu_production_management.click()
        logger.info("点击二级菜单【生产计划】")
        self.menu_production_plan.click()
        logger.info("点击三级菜单【生产订单】")
        self.menu_production_order.click()
        logger.info("点击页面入口文本【生产订单】进入功能页")
        self.entry_production_order.click()
        self.page.wait_for_load_state('networkidle')
```



2.4 录制页面



3. 脚本执行（命令行）

3.1 命令行执行入口

[run.py](#)

3.2 参数解析

- 1 --project 非必填参数 代表项目名 简写'-p'
- 2 --module 非必填参数 代表模块名 简写'-m'
- 3 --case 非必填参数 代表用例名 简写'-c'
- 4 --url 非必填参数 代表项目执行URL，不传则从配置文件获取 简写'-u'
- 5 --username 非必填参数 代表登录用户名，不传则从配置文件获取 简写'-user'
- 6 --pwd 非必填参数 代表登录密码，不传则从配置文件获取 简写'-pwd'
- 7 --wh 非必填参数 代表是否推送webhook（0=不推送，1=推送，默认1 简写'-w'

3.3 执行示例

单用例执行：仅执行指定用例，如base_record

```
1 # 单用例执行
2 python run.py -p LX -m scenario_case -c base_record
```

模块用例执行：执行指定模块下所有用例，如scenario_case模块下的所有用例

```
1 # 模块用例执行
2 python run.py -p LX -m scenario_case
```

项目用例执行：执行指定项目下所有用例，如LX项目下所有用例

```
1 # 项目用例执行
2 python run.py -p LX
```

推送用例执行：会将执行结果推送到指定企业微信群

```
1 # 单用例执行
2 python run.py -p LX -m scenario_case -c base_record -w 1
```

自动化测试执行结果

执行项目: 灵犀
执行模块: 场景测试
用例名称: 场景用例: 测试正向生产流程
执行结果: **✖ 失败**
用例总数: 1
成功用例: 0
失败用例: 1
开始时间: 2026-06-09 08:58:49
结束时间: 2026-06-09 09:07:03
执行时长: 493秒
用例平台: [点击查看](#)

4. 用例管理平台

4.1 用例列表

测试用例管理

用例列表

执行记录

版本管理

用例列表

当前版本: V3

共 68 条用例

手动同步用例

更新用例

批量执行

模块

刷新

灵犀

生产订单 24 条

场景测试 8 条

生产工单 36 条

模块选择

灵犀 - 场景测试

共 8 条用例

用例列表

批量执行用例

1

项目: 灵犀 模块: 场景测试

执行

2

项目: 灵犀 模块: 场景测试

执行

3

项目: 灵犀 模块: 场景测试

执行

create_mat

项目: 灵犀 模块: 场景测试 场景: 场景用例: 创建物料信息

执行

material_preparation_proc

项目: 灵犀 模块: 场景测试 场景: 场景用例: 测试替代料流程

执行

merge_production_proc

项目: 灵犀 模块: 场景测试 场景: 场景用例: 测试订单合并生产流程

执行

positive_production_proc

项目: 灵犀 模块: 场景测试 场景: 场景用例: 测试正向生产流程

执行

replace_materials_proc

项目: 灵犀 模块: 场景测试 场景: 测试替代料流程

执行

单用例执行按钮

4.2 执行记录

测试用例管理

用例列表

执行记录

版本管理

执行记录

共 6 条记录

点击可以查看执行报告

ID	项目	模块	用例	版本	开始时间	结束时间	用例数	通过	失败	状态	操作
6	灵犀	场景测试	场景用例: 创建物料信息	V3	2026-06-08 13:19:15	2026-06-08 13:23:56	1	0	1	失败	查看报告
5	灵犀	生产订单	测试生产订单自动拆分日期规则	V2	2026-06-02 16:07:08	2026-06-02 16:08:12	1	1	0	已完成	查看报告
4	灵犀	生产订单	测试生产订单自动拆分日期规则	V2	2026-06-02 15:58:17	2026-06-02 15:58:19	0	0	0	失败	查看报告
3	灵犀	生产订单	测试生产订单自动拆分日期规则	V2	2026-06-02 15:41:50	2026-06-02 15:42:51	1	1	0	已完成	查看报告
2	LD	base_information	verify_warehouse_create_update	V2	2026-06-02 15:40:12	2026-06-02 16:14:47	0	0	0	已超时	查看报告
1	灵犀	生产订单	测试生产订单自动拆分日期规则	V2	2026-06-02 15:29:30	2026-06-02 15:30:32	1	1	0	已完成	查看报告

状态分为: 成功, 失败, 超时和进行中

4.3 测试报告

自动化测试报告

报告时间: 2026-06-08 13:19:15

返回执行记录

执行概况

1

执行总数

0

通过

1

失败

0.0%

通过率

测试环境

环境地址: http://10.30.22.45:8080/

执行用户: admin

测试版本: V3

开始时间: 2026-06-08 13:19:15

结束时间: 2026-06-08 13:23:56

总耗时: 281.0s

平台: Windows

测试执行列表

展开后可以查看错误信息, 日志, 截图和录屏

#	结果	模块	测试用例名称	用例场景	详情
1	失败	场景测试	create_mat	场景用例: 创建物料信息	展开

执行详情

4.4 执行结果记录

日志: 所有都会有日志记录, 可以在页面或者logs文件夹中查看

截图: 用例执行出错或者页面后台报404等错误码时会截图

视频: 用例执行时会自动录屏, 若最终执行成功则会自动删除录屏节省空间

#	结果	模块	测试用例名称	用例场景	详情
1	失败	场景测试	create_mat	场景用例: 创建物料信息	展开

错误信息:

src\testcase\lx\scenario_case\test_create_mat.py:162: in test_create_mat raise AssertionError("生效sop失败") E AssertionError: 生效sop失败

截图:

20260608_132323_881984_page_error_test_create_mat_status404.png

20260608_132323_873949_page_error_test_create_mat_status404.png

20260608_132224_487126_page_error_test_create_mat_status404.png

20260608_132224_470923_page_error_test_create_mat_status503.png

20260608_132223_971522_page_error_test_create_mat_status404.png

20260608_132223_966321_page_error_test_create_mat_status404.png

20260608_132223_816379_page_error_test_create_mat_status404.png

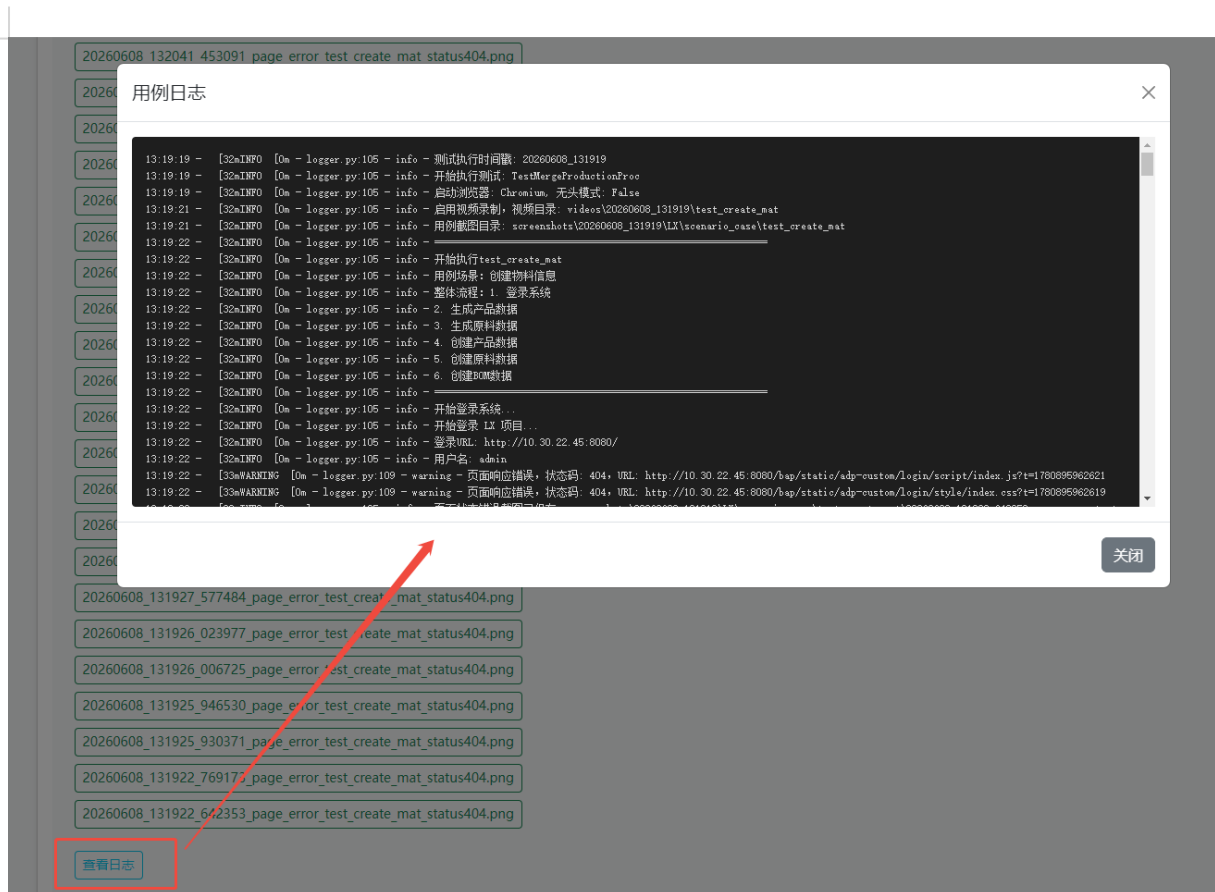
视频:

25364e108564cc77bdfb8aea938e3e31.webm

4d608bc377c0ad01c0d926c2bfcd24a.webm

1329e625bd15ab1a35dbbac5eab64fef.webm

截图和视频可以直接点击查看



4.5 版本控制

情况分析：产品版本不同，页面可能存在差异，如果脚本随着产品一起迭代，则无法兼容旧版本，故引入版本控制功能，用户可以根据环境中产品的版本切换脚本的版本

详细设计查看项目代码中的版本控制使用说明.md文档

